

1. De la tastatură se citesc cuvinte. Să se creeze o listă dublu înlănțuită ordonată alfabetic, care conține în noduri cuvintele distincte și frecvența lor de apariție. Se va afișa conținutul listei în ordine alfabetică.
2. Elementele unei mulțimi M sunt notate cu litere mici din alfabet. Se citesc perechi de elemente x, y (x, y aparțin mulțimii M) cu semnificația că elementul x precede elementul y . Să se afișeze elementele mulțimii M într-o anumită ordine, încât pentru oricare elemente x, y cu proprietatea că x precede pe y , elementul x să fie afișat înaintea lui y .
3. Să se scrie programul care creează două liste ordonate crescător după o cheie numerică și apoi le interclasează.
4. Să se conceapă o structură dinamică eficientă pentru reprezentarea matricelor rare. Să se scrie operații de calcul a sumei și produsului a două matrici rare. Afișarea se va face în formă naturală.
5. Să se conceapă o structură dinamică eficientă pentru reprezentarea în memorie a polinoamelor. Se vor scrie funcții de calcul a sumei, diferenței, produsului și împărțirii a două polinoame.
6. Se citește de la tastatură o expresie postfixată corectă sintactic. Să se scrie programul de evaluare a sa. Expresia conține variabile formate dintr-o literă și operatorii binari $+$, $-$, $*$, $/$.
7. Să se implementeze un buffer circular, care conține înregistrări cu datele unui student și asupra căruia acționează principiul de sincronizare producător-consumator, care constă în următoarele:
 - ◇ înregistrările sunt preluate în ordinea producerii lor;
 - ◇ dacă bufferul nu conține înregistrări, consumatorul este întârziat până când producătorul depune o înregistrare;
 - ◇ dacă bufferul este plin, producătorul este întârziat până când consumatorul a preluat o înregistrare.
8. Să se simuleze următorul joc: n copii, ale căror nume se citesc de la tastatură, stau în cerc. Începând cu un anumit copil (numele său se citește), se numără copiii în sensul acelor de ceasornic. Fiecare al m -lea copil (m se citește) iese din joc. Numărătoarea continuă începând cu următorul copil din cerc. Câștigă jocul ultimul copil rămas în cerc.

9. Să se evalueze un arbore care conține în noduri constantele 0 și 1 și operatorii AND, OR, NOT. Să se scrie o funcție care determină dacă doi arbori binari sunt echivalenți (arborii binari sunt echivalenți dacă sunt structural echivalenți și datele corespunzătoare nodurilor sunt aceleași).
10. Să se scrie un program care transformă un arbore binar într-o listă dublu înlănțuită și care folosește funcții de pretty-print (tipărire frumoasă) pentru afișare.
11. Arborele genealogic al unei persoane se reprezintă astfel: numele persoanei este cheia nodului rădăcină și pentru fiecare nod cheia descendentului stâng este numele tatălui, iar a descendentului drept este numele mamei. Se citesc două nume de la tastatură. Ce relație de rudenie există între cele două persoane? Se presupune că o familie are doar un singur fiu.
12. Să se scrie un program care pentru un arbore binar:
- ◇ interschimbă subarboarele drept cu cel stâng pentru un nod dat;
 - ◇ determină înălțimea;
 - ◇ determină numărul de frunze.
13. Să se scrie un program de construire și traversare a unui arbore oarecare.
14. Să se verifice dacă operația de ștergere a unui nod dintr-un arbore de căutare este comutativă (ștergerea nodurilor x și y se poate face în orice ordine).
15. Să se implementeze operația de interclasare a doi arbori de căutare.
16. Se consideră un arbore de căutare care conține elemente cu informația utilă de tip șir de caractere. Să se scrie o funcție de căutare, inserare și ștergere a șirului de caractere permițându-se folosirea șabloanelor, spre exemplu * pentru orice subșir sau ? pentru orice caracter.
17. Să se scrie o funcție care să verifice dacă există o cale între două noduri date (v , w) ale unui graf orientat $G=(V, E)$.

18. Pentru un graf neorientat dat $G=(N, R)$, să se scrie o funcție care să verifice dacă graful este conex sau nu.
19. Să se implementeze algoritmul lui Floyd de găsire a căilor de cost minim din oricare vârf al unui graf neorientat. Se vor afișa căile de cost minim între două vârfuri, date de la tastatură.
20. Să se implementeze algoritmul lui Kruskal de găsire a arborelui de acoperire a unui graf neorientat.
21. Se va crea o tabelă fixă cu cuvintele rezervate din limbajul C. Se va scrie apoi o funcție de căutare binară a unui cuvânt în tabelă.
22. Să se afișeze frecvența de apariție a literelor dintr-un text utilizând o tabelă de dispersie.
23. Se dau m vectori $V_1, V_2, \dots, V_m$, care conțin $n_1, n_2, \dots, n_m$ elemente, ordonate crescător după o cheie. Se interclasează vectorii dați, obținându-se un vector de lungime $n_1+n_2+\dots+n_m$ elemente, ordonate crescător. Se știe că interclasarea a doi vectori care conțin n_1 , respectiv n_2 elemente necesită un timp proporțional cu suma lungimilor lor. Să se determine ordinea optimă în care trebuie efectuată interclasarea tuturor vectorilor dați.
24. Problema rucsacului. Greutatea maximă care poate fi transportată cu un rucsac este G . Dându-se n materiale, fiecare având greutatea m și costul C pe unitatea de greutate, să se găsească ce cantitate din fiecare material să fie introdus în rucsac pentru ca să se obțină câștigul maxim. Se vor deosebi două cazuri:
- a) un material poate fi luat numai în întregime;
 - b) se poate lua o fracțiune din material.
25. Problema activităților. Există o mulțime $S=1, 2, 3, \dots, n$ de n activități care doresc să folosească o aceeași resursă (de exemplu o sală de clasă). Această resursă poate fi folosită de o singură activitate la un anumit moment de timp. Fiecare activitate i are un timp de pornire tp_i și un timp de terminare tf_i ($tp_i < tf_i$). Dacă este selectată activitatea i , ea se desfășoară pe durata $[tp_i, tf_i)$. Spunem că activitățile i și j sunt compatibile dacă duratele lor nu se intersectează. Să se selecteze o mulțime maximală de activități mutual compatibile.

26. Colorarea grafurilor. Fiind dat un graf neorientat $G=(X, \Gamma)$, unde X este mulțimea formată din n noduri, iar Γ este mulțimea muchiilor și un număr de m culori, se cere să se determine toate colorările posibile ale nodurilor grafului folosind cele m culori, astfel încât oricare două noduri adiacente să fie colorate în mod diferit.
27. Problema ciclului hamiltonian. Se dă un graf conex neorientat $G=(X, \Gamma)$ prin matricea costurilor pozitive. Se cere să se determine ciclul hamiltonian de cost minim (ciclul hamiltonian este un ciclu simplu care trece prin toate nodurile).
28. Fiind dată o matrice A de $n \times n$ elemente numere naturale, să se determine cea mai mică sumă de n elemente luate din linii diferite și coloane diferite.
29. Fiind date o tablă de șah de dimensiune $n \times n$ pătrate și un cal plasat în pătratul din stânga sus al tablei, să se afișeze toate posibilitățile de mutare a calului astfel încât să treacă o singură dată prin fiecare pătrat al tablei.
30. Un labirint este codificat printr-o matrice de $n \times m$ elemente ale cărei culoare sunt reprezentate prin elemente egale cu 1, situate în poziții consecutive pe o aceeași linie sau coloană, celelalte elemente fiind 0. O persoană se găsește în poziția (i, j) din interiorul labirintului. Se cere afișarea tuturor traseelor de ieșire din labirint care nu trec de mai multe ori prin același loc.
31. Se consideră o mulțime formată din n elemente numere întregi. Să se genereze toate submulțimile acestei mulțimi având proprietatea că suma elementelor lor este egală cu S .
32. Există următorul joc: pe o linie de cale ferată se află n vagoane unul lângă altul, numerotate cu valori distincte din mulțimea $1 \dots n$. O macara poate lua k vagoane de pe linie și le poate așeza în partea dreaptă, la sfârșitul șirului de vagoane, care apoi prin împingere ajung din nou unul lângă altul, în noua ordine creată după operația respectivă. Dându-se ordinea inițială a vagoanelor, se cere să se determine (dacă este posibil) numărul minim de operații pe care trebuie să le efectueze macaraua pentru ca în final vagoanele să se afle în ordine crescătoare $1, 2, \dots, n$.

33. Jocul PERSPICO. 15 plăcuțe pătrate sunt încadrate într-un cadru de dimensiune 4×4 , o poziție fiind liberă. Orice plăcuță vecină cu această poziție liberă poate fi mutată în locul ei. Cele 15 plăcuțe sunt numerotate de la 1 la 15. Se începe dintr-o stare inițială, care corespunde unei distribuții oarecare a celor 15 plăcuțe și a locului liber în cele 16 poziții posibile. Problema constă în a trece, folosind mutări posibile, din starea inițială în starea finală.

1		3	4
5	2	7	8
9	6	10	11
13	14	15	12

Exemplu de
configurație
inițială

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Configurație
finală

34. Pe malul unui râu se află $2n$ băștinași din care n sunt canibali. Aceștia doresc să traverseze râul utilizând o barcă care poate transporta cel mult k persoane. Dacă pe un mal sau în barcă sunt mai mulți canibali decât ceilalți, atunci canibalii îi vor mânca. Cum vor reuși să treacă toți pe malul opus fără să se mănânce și fără a apela la alte persoane.
35. Pe un pod se află n capre care vin dintr-un sens, cu n capre care vin din sens opus. Acestea nu se pot ocoli, însă fiecare capră poate sări peste o singură capră din grupul opus și desigur poate avansa dacă în fața sa este un spațiu liber. Cum reușesc aceste capre să traverseze podul doar prin cele două mișcări posibile (avans și săritură).
36. Fiind dat un vector ce conține elemente de tip întreg ordonate crescător, să se scrie o funcție de căutare a unui element dat în vector, returnându-se poziția sa.
37. Problema turnurilor din Hanoi. Se dau trei tije. Pe una dintre ele sunt așezate n discuri de mărimi diferite, discurile de diametre mai mici fiind așezate peste discurile cu diametre mai mari. Se cere să se mute aceste discuri pe o altă tijă, utilizând tija a treia ca intermediar, cu condiția mutării a câte unui singur disc și fără a pune un disc de diametru mai mare peste unul cu diametru mai mic.
38. La un concurs de tir, ținta este alcătuită din cercuri concentrice, numerotate din exterior spre interior. Fiecărui sector determinat de două cercuri succesive îi este atașată o valoare strict pozitivă, reprezentând punctajul primit de concurent în cazul lovirii acestui sector. Să se determine numărul minim de lovituri pe care trebuie să le execute un concurent pentru a obține exact k puncte.

39. Se dau două numere naturale A și B și un vector v care conține n numere naturale. Să se determine dacă se poate trece din A în B , știind că singurele operații permise sunt:

- a) Adunarea la A a oricâte numere din vectorul v ;
- a) Scăderea din A a oricâte numere din vectorul v .

Fiecare număr poate fi adunat, respectiv scăzut de mai multe ori.

Dacă răspunsul la întrebare este afirmativ, se cere numărul minim de operații prin care se poate trece din A în B .

40. Pe o creangă de măr, se află n mere, fiecare caracterizat prin distanța h_i în cm de la pământ până la poziția în care se află și prin greutatea sa g_i în grame. Un culegător dorește să culeagă o cantitate exprimată în grame cât mai mare de mere. După ce este cules un măr întreaga creangă devine mai ușoară și se ridică în sus cu x cm. Culegătorul ajunge doar la merele aflate la o înălțime mai mică sau egală cu d cm. Să se determine greutatea maximă de mere care poate fi culeasă și ordinea în care sunt culese merele. Se citesc: n, x, d și $(g_i, h_i) i=1...n$.

41. Să se găsească maximul unei funcții $f(x)$ în intervalul $[a, b]$.

42. Se dă un graf neorientat cu n noduri. Se cere să se determine numărul minim de culori necesare pentru a colora nodurile grafului dat, astfel încât două vârfuri legate printr-o muchie să fie colorate cu culori diferite.

43. Se dă un graf neorientat cu n noduri. Se cere să se determine o submulțime maximă de noduri cu proprietatea că oricare două noduri din ea nu sunt legate printr-o muchie.

44. Descrieți algoritmul de sortare prin inserare, la care modificați căutarea liniară cu o căutare binară (în cadrul subtabloului din stânga elementului curent). Devine acest algoritm mai performant?

45. Adaptați algoritmul quick-sort pentru a determina într-un șir de lungime n cu elemente întregi, al m -lea mai mic element.

46. Să se sorteze un număr de n elemente numerotate de la 1 la n caracterizate prin anumite relații de precedență notate (j, k) , având semnificația că elementul j precede ca ordine elementul k . Sortarea trebuie să aibă ca rezultat o listă în care oricare element k este devansat de predecesorul său (sortarea topologică).

47. Să se descrie algoritmul care realizează următoarele acțiuni specifice unui examen de admitere:
- efectuarea calculului mediilor candidaților la examenul de admitere;
 - repartizarea celor admiși după opțiuni (se consideră că există m opțiuni, fiecare cu un număr dat de candidați admiși) și afișarea listei;
 - afișarea în ordinea descrescătoare a mediilor a tuturor candidaților neadmiși.
- Se consideră că examenul constă din două probe, la medii egale (calculate cu două zecimale, prin trunchiere), departajarea se face după nota la prima probă (dacă egalitatea se menține, se ia în considerare a doua), admitându-se depășirea numărului de locuri în caz de egalitate și după acest criteriu.
48. Există n puncte: $p_1, p_2, \dots, p_n$, toate localizate pe axa X , iar x_i este coordonata lui p_i . Se presupune ca $x_1=0$ și punctele sunt numerotate de la stânga la dreapta. Aceste n puncte determină $n*(n-1)/2$ distanțe (de valori nu neapărat unice), $d_1, d_2, \dots, d_m$ între oricare 2 puncte distincte x_i și x_j . Problema reconstrucției este următoarea: se dau m valori, (sortate în ordine crescătoare), care reprezintă setul de distanțe între $m=n*(n-1)/2$ perechi de puncte. Se cere să se reconstituie coordonatele celor n puncte.
49. Să se scrie un program care să găsească toate variantele, precum și cea optimă pentru tăierea unui fir de lungime L în părți de lungimi $L_1, L_2, \dots, L_n$ date, în condițiile:
- a) nu există nici un fel de restricție în ce privește numărul de bucăți din fiecare lungime;
 - b) se taie cel mult o bucată de fiecare lungime;
 - c) numărul de bucăți de fiecare lungime să difere prin cel mult o unitate.
50. Se consideră următoarea problemă de formatare a textului unui paragraf, care este o secvență de n cuvinte, $c_1, c_2, \dots, c_n$, de lungimi $a_1, a_2, \dots, a_n$, care trebuie afișate pe linii de lungime L . Cuvintele trebuie separate prin spații, a căror lungime ideală este b , dar spațiile pot fi extinse sau comprimate (după comprimare trebuie să rămână totuși > 0), astfel încât o linie ce conține cuvintele $c_i c_{i+1} \dots c_j$ să ocupe exact lungimea L . Pentru fiecare spațiu b' care este mai mare sau mai mic decât lungimea ideală b , se definește un "grad de urâtenie" ca fiind $|b' - b|$. Pentru întregul paragraf, este de dorit ca suma acestor grade de urâtenie să fie minim. Se cere să se scrie programul care determină valorile lungimilor spațiilor separatoare astfel încât să se obțină o formatare cât mai estetică a textului.
51. Să se scrie câte o procedură recursivă pentru:
- a) transformarea unui întreg din baza 10 în altă bază dată;
 - b) tipărirea nodurilor unei liste în ordine inversă;
 - c) analog cu b), dar pentru elementele unui tablou;
 - d) copierea în ordine inversă a liniilor dintr-un fișier text, în altul.

52. Se consideră un set de N întrebări, fiecare având un punctaj P_i . Să se genereze toate chestionarele conținând un număr de întrebări între A și B și având un punctaj total între C și D .

53. Se consideră un triunghi format din n linii, pe fiecare linie i sunt i numere întregi pozitive, ca în exemplul de mai jos:

```
      7
     3  8
    8  1  0
   2  7  4  4
  4  5  2  6  5
```

Să se scrie un program care calculează cea mai mare sumă a numerelor aflate pe un drum care leagă vârful de sus al triunghiului cu baza. Drumul este astfel construit încât la fiecare pas se coboară pe diagonală, spre stânga sau spre dreapta.

Exemplu: pentru triunghiul de mai sus, drumul căutat este: 7 - 3 - 8 - 7 - 5

54. Se consideră o suprafață caroiată de dimensiune $n \times n$, în care fiecare pătrat are una din culorile: galben, roșu, albastru sau verde. Configurația suprafeței se citește de la tastatură (sau dintr-un fișier). Să se determine și să se tipărească dacă există drumuri (minime) între colțurile opuse - un drum trebuie să cuprindă doar pătrate de aceeași culoare; deplasarea dintr-un pătrat se poate face în oricare din cei opt vecini.

55. Pe un circuit electric trebuie plasate n componente. Plasarea lor se face în n poziții bine determinate, identificate cu numerele $1..n$. Se dă o matrice c în care $c[i,j]$ reprezintă numărul de conexiuni care trebuie făcute între componentele i și j , precum și o matrice d , cu $d[p,q]$ reprezentând distanța între punctele p și q . Cablarea constă în plasarea fiecărei componente într-o anumită poziție. Nu se pot așeza 2 componente în aceeași poziție. Costul cablării este suma valorilor $c[i,j] \cdot d[p,q]$, unde componenta i a fost plasată în poziția p iar componenta j în poziția q .

Se cere să se determine o plasare a componentelor pe poziții astfel încât costul total al cablării să fie minim.

56. Între n orașe există o rețea de drumuri care permite ca dintr-un oraș să se ajungă în oricare din celelalte. Între 2 orașe există cel mult un drum direct, de lungime cunoscută, iar timpul necesar pentru parcurgerea unui drum este proporțional cu lungimea sa. Se cere să se realizeze programul pentru determinarea traseului pe care se poate merge între două orașe date, într-un timp minim.

57. Să se scrie un program care să vină în sprijinul alcătuirii orarului cursurilor facultative. Există un număr de $N=20$ cursuri și $M=100$ studenți. La începutul anului, fiecare student își exprimă opțiunea privind cursurile la care dorește să participe. Un student poate alege un număr oarecare de cursuri ($0 \dots N$), dar în medie numărul cursurilor alese este de 2-3. Problema responsabilului cu alcătuirea orarului este de a identifica care sunt cursurile care pot fi programate a se desfășura în paralel. Se pot desfășura în paralel 2 sau mai multe cursuri, cu condiția să nu existe suprapuneri în orarul nici unui student.

Exemplu:

$N=5$, $M=6$

studentul 1 alege cursurile 1,2

studentul 2 nu alege nimic

studentul 3 alege cursurile 1,3

studentul 4 alege cursul 4

studentul 5 alege cursul 4

studentul 6 alege cursul 5

Cursurile care se pot desfășura în paralel sunt: (1,4,5) (2,3)

58. Să se implementeze un TDA "numere mari". Un "număr mare" este un întreg care poate avea până la 200 de cifre. Pentru reprezentarea internă a numerelor se vor folosi șiruri de caractere. Asupra acestor numere se definesc operațiile aritmetice (adunare, scădere, înmulțire).

Să se scrie un program care citește 2 numere și realizează operațiile aritmetice asupra lor.

59. Un participant la un joc de noroc pornește având la start o sumă de bani A . La fiecare tură a jocului, jucătorul poate pierde sau câștiga o sumă în valoare fixă B . Să se găsească toate posibilele variante de desfășurare a jocului (secvențe pierdere - câștig), astfel încât după N runde, jucătorul să aibă aceeași sumă de bani, A , ca la start.

60. Un șofer dorește să conducă din orașul A în orașul B , între care distanța este de $n * 10$ km (n număr întreg ≥ 1). Începând cu punctul de plecare A (inclusiv) există benzinării (numerotate începând cu 0) la fiecare 10 km. Mașina șoferului consumă 1 litru de benzină la fiecare 10 km și are o capacitate a rezervorului de c litri (c număr întreg ≥ 1). Șoferul are la dispoziție o hartă în care este trecut prețul la fiecare benzinărie.

Să se scrie un program care să indice de unde și în ce cantitate trebuie să cumpere șoferul benzină pentru a parcurge drumul cu cost minim, și care este acest cost. Programul va afișa o singură soluție. Inițial mașina nu are benzină în rezervor, iar de la o benzinărie șoferul poate cumpăra orice cantitate de benzină, în limitele capacității rezervorului.

Exemplu:

Fișierul de intrare:

5 2

2.9 3.1 2.8 3.3 2.9

Ieșirea programului:

Cumpără 2 l de la benzinăria 0

Cumpără 2 l de la benzinăria 2

Cumpără 1 l de la benzinăria 4

Cost total: 14.3

61. Imaginea unui mozaic se află într-un fișier text, fiecare caracter desemnând culoarea unei plăcuțe din mozaic. Pentru simplitate, toate plăcuțele sunt de formă pătrată, de aceeași dimensiune, și nu sunt decât două culori, reprezentate în fișier prin caracterele 0 și 1. Toate liniile fișierului au aceeași lungime și nu depășesc 250 caractere. Figurile din imaginea mozaicului sunt formate din grupe de 1 sau mai multe pătrățele adiacente, de culoare 1. Plăcuțele adiacente au cel puțin o latură comună cu restul grupului. Scrieți un program care să ceară, prin dialog de la consolă, numele unui fișier (dimensiunea fișierului poate fi oricât de mare - peste 100 de linii), și să afișeze numărul figurilor găsite în acel fișier.

62. La orele serii, piața centrală în formă de hexagon regulat a orașului X abundă de vizitatori. Atracția o constituie statuile plasate la distanțe egale și luminate fiecare de un reflector. La încheierea programului de vizitare, sarcina lui Gogu este de a stinge toate reflectoarele, astfel încât pierderea de energie să fie minimă. El pornește din cabina sa, notată 0, aflată lângă colțul din stânga-sus al pieții, revenind în același punct după stingerea tuturor reflectoarelor. Se poate deplasa numai pe aleile ce leagă cabina de prima statuie, respectiv statuile între ele. Aleile sunt de lungimi egale, iar parcurgerea uneia se face într-o unitate de timp. Poate să treacă de mai multe ori pe lângă aceeași statuie, la prima trecere stingând reflectorul. Pregătiți traseul lui Gogu! Dacă există două variante cu aceeași energie minimă consumată, se va considera soluția care se parcurge într-un timp mai scurt. Ca date de intrare, se citesc din fișierul IN.TXT întregii: pe prima linie numărul N al statuiilor de pe o latură a pieții, inclusiv colțurile, iar pe linia următoare consumurile de energie pe unitatea de timp ale reflectoarelor, în ordinea parcurgerii lor pe linii, de la stanga la dreapta. Se vor afișa: timpul T de parcurgere a traseului, pierderea de energie E și traseul urmat (succesiunea statuiilor pe lângă care se trece).

